

Adaptive and Policy-Aware DoS Detection and Mitigation in SDN

Network & Cloud Infrastructures - Security Project Work - a.a. 2025/2026

De Rosa - DE9000164

Giaquinto - DE9000142

Repository: <https://github.com/NCIS-unina/ncis-project-work-2026-derosa-digennaro-giaquinto-gismondi>

Di Gennaro - DE9000051

Gismondi - DE9000188

1. Obiettivo e contesto

Il progetto parte da un prototipo SDN per la rilevazione e mitigazione reattiva di attacchi DoS volumetrici e ne sviluppa una versione più completa. La rete è emulata con Mininet/Open vSwitch, mentre Ryu controlla gli switch tramite OpenFlow 1.3.

Lo scopo del progetto è verificare in modo riproducibile come un controller SDN possa apprendere il traffico normale, individuare la sorgente che genera il carico anomalo, applicare un blocco selettivo e rimuoverlo quando il traffico torna normale. In parallelo, il controller deve poter applicare anche decisioni amministrative esterne.

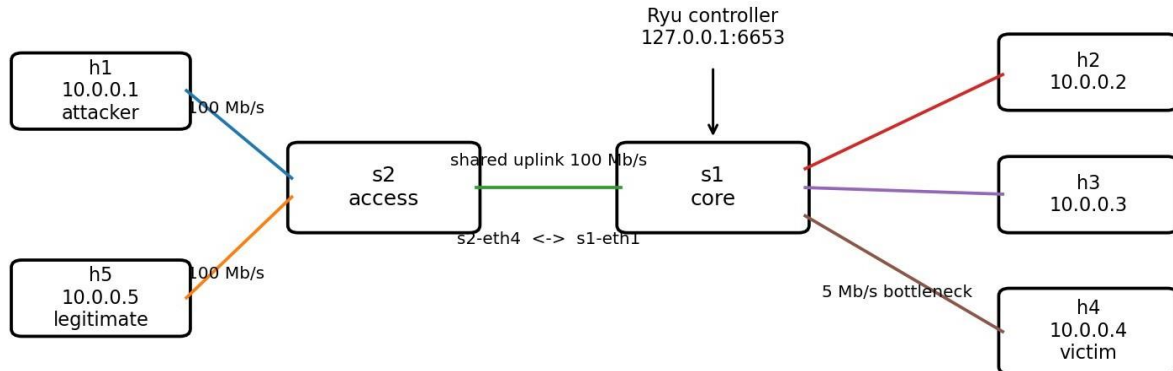


Figura 1 - Topologia finale a due switch. h1 e h5 condividono l'uplink s2-s1; h4 è la vittima dietro un bottleneck da 5 Mbit/s.

Questa topologia ci permette di riprodurre chiaramente il problema dell'over-blocking: un DROP generico sull'uplink s1-eth1 bloccherebbe sia l'attaccante h1 sia il client legittimo h5.

2. Criticità affrontate e architettura finale

Fase	Flaw / feature	Soluzione
M1	Lack of Modular Design	Controller diviso in moduli separati per monitoraggio, detection, tracking delle sorgenti, mitigazione, logging e policy.
M2	Over-blocking	DROP mirato all'IP sorgente invece di bloccare tutta la porta condivisa.
M3	Static Threshold	Training iniziale, baseline EWMA e soglia adattiva con valore minimo.
M4	Controller-Centric Decisions	Blocklist esterna in JSON, tradotta dal controller in regole OpenFlow persistenti.
M5	Inflexible Block/Unblock	Rimozione del DROP in base al traffico osservato, senza timeout fisso.
M6	External Allowlist	Le sorgenti in allowlist vengono ancora monitorate, ma non sono bloccate automaticamente.

Per rendere il controller più semplice da modificare e testare, abbiamo separato le responsabilità principali. Il monitor raccoglie le misure, il detector decide quando il traffico è anomalo, il tracker associa IP/MAC alle porte, il selector individua la sorgente candidata e il mitigator installa o rimuove le regole OpenFlow. Un modulo separato gestisce invece le policy amministrative esterne.

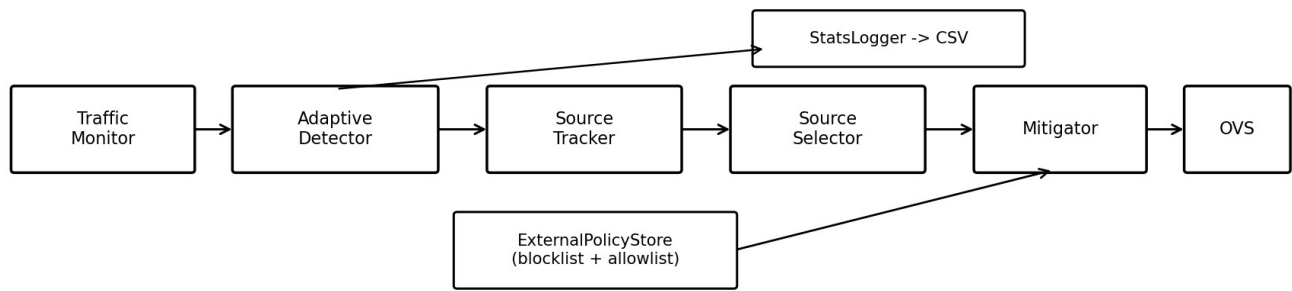


Figura 2 - Pipeline logica della versione finale.

Regole di mitigazione e policy

- Mitigazione automatica: priority=100, match su (in_port, IPv4, ipv4_src), hard_timeout=0 e idle_timeout=0.
- Blocklist amministrativa: priority=110 e regola persistente finché l'IP resta in blocked_ipv4. Quando possibile, il DROP viene installato sulla porta host-facing più vicina alla sorgente.
- Se un IP compare contemporaneamente in blocklist e allowlist, prevale la blocklist amministrativa.
- Se il file JSON è temporaneamente non valido durante un salvataggio, il controller mantiene le policy già installate invece di rimuoverle.

3. Detection adattiva e intelligent unblock

Parametro	Valore	Ruolo
POLL_INTERVAL	2 s	Campionamento PortStats
MIN_THRESHOLD_MBPS	1.5 Mbit/s	Floor della soglia
THRESHOLD_MULTIPLIER	1.5	Moltiplicatore baseline
EWMA_ALPHA	0.2	Aggiornamento baseline
TRAINING_SAMPLES	5	Campioni benigni attivi
BASELINE_UPDATE_MIN_RATIO	0.25	Ignora residui/idle post-training
REQUIRED_HITS	3	Conferma dell'anomalia
UNBLOCK_RATE_MBPS	0.1 Mbit/s	Soglia di quiete
UNBLOCK_QUIET_SAMPLES	3	Campioni quiet per DELETE_STRICT

Algoritmo

Durante il training, la baseline è calcolata come media dei campioni di traffico attivo. Terminato il training, la soglia è:

```
threshold = max(1.5, 1.5 * baseline)
```

Se il rate è sotto soglia, la baseline può essere aggiornata con EWMA:

```
baseline_new = 0.2 * rate + 0.8 * baseline_old
```

Durante il traffico sospetto la baseline non viene aggiornata. Inoltre, dopo il training ignoriamo i campioni con rate inferiore a $\max(0.1 \text{ Mbit/s}, 25\% \text{ della baseline})$, così i periodi idle e i piccoli residui di fine flusso non abbassano artificialmente la soglia.

Consideriamo un attacco confermato dopo tre campioni consecutivi sopra soglia. Il SourceSelector sceglie la sorgente non presente in allowlist che sta generando il traffico più elevato e il Mitigator installa un DROP mirato per `ipv4_src`. Non usiamo più un `hard_timeout` fisso di 20 s: il controller continua a osservare la porta host-facing della sorgente bloccata e rimuove la flow con `OFFPFC_DELETE_STRICT` dopo tre campioni consecutivi $\leq 0.1 \text{ Mbit/s}$.

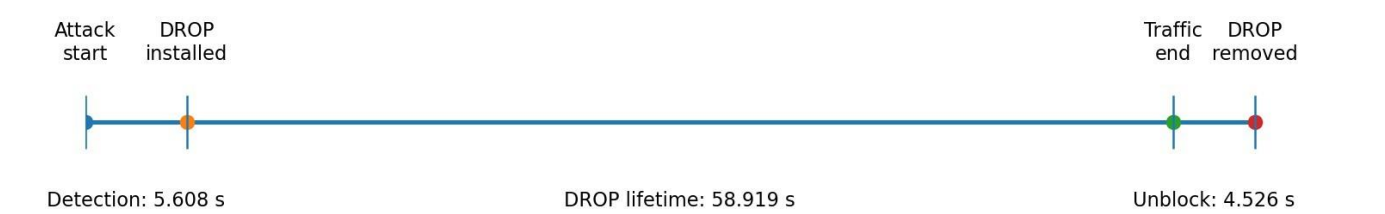


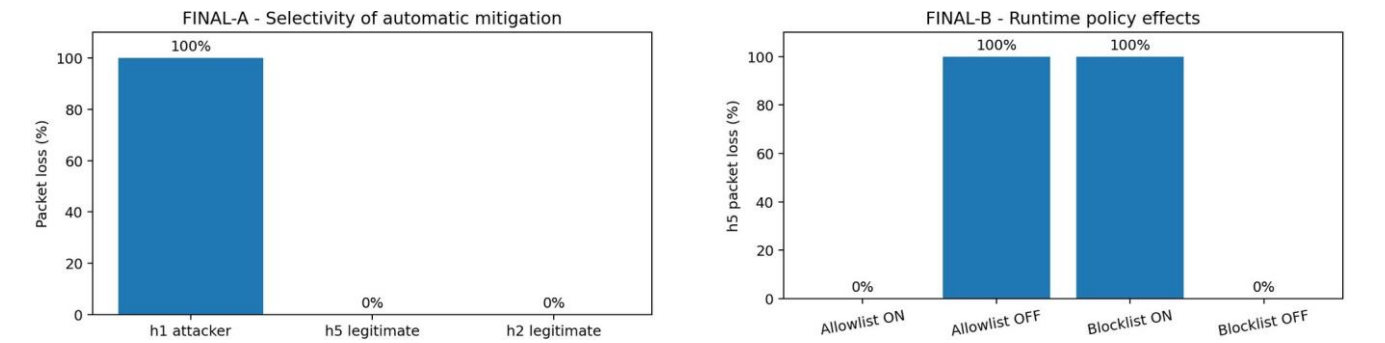
Figura 3 - Timeline di FINAL-A. Il DROP resta attivo per tutta la fase offensiva e viene rimosso dopo la quiete osservata.

Policy esterne

Il file `policy/blocklist.json` contiene `blocked_ipv4` e `allowlisted_ipv4` e può essere modificato mentre il controller è in esecuzione. Il file viene riletto ogni secondo. Gli IP in `blocked_ipv4` ricevono un DROP amministrativo persistente (`priority=110`); gli IP in `allowlisted_ipv4` continuano a essere monitorati, ma la mitigazione automatica viene soppressa e viene registrato l'evento `ALLOWLIST_SUPPRESSED`.

4. Validazione sperimentale

Per la validazione finale abbiamo eseguito due esperimenti end-to-end sulla versione completa del controller. FINAL-A verifica soglia adattiva, mitigazione mirata e sblocco automatico; FINAL-B verifica allowlist e blocklist a runtime. Prima di iniziare i test abbiamo controllato la connettività con pingall, ottenendo 0% di pacchetti persi (20/20 ricevuti).



FINAL-A - risultati chiave

Baseline	Threshold	Detection delay	Unblock dopo fine traffico
2.044 Mbit/s	3.066 Mbit/s	5.608 s	4.526 s

In FINAL-A abbiamo osservato la regola priority=100, in_port=1, nw_src=10.0.0.1, actions=drop. Durante la mitigazione h1 ha avuto 100% packet loss, mentre h5 e h2 sono rimasti a 0%. La regola è rimasta installata per 58.919 s, quindi ben oltre il vecchio timeout fisso di 20 s, ed è stata rimossa circa 4.526 s dopo la fine effettiva del traffico offensivo.

FINAL-B - risultati chiave

In FINAL-B, con h5 in allowlist e traffico ad alto rate, il controller ha registrato ALLOWLIST_SUPPRESSED senza installare una flow priority=100; h5 è rimasto raggiungibile con 0% packet loss. Dopo aver rimosso h5 dall'allowlist, lo stesso traffico ha prodotto DROP_INSTALLED e 100% packet loss per h5, mentre h1 e h2 sono rimasti raggiungibili. Inserendo h5 in blocked_ipv4 abbiamo osservato su s2 la regola priority=110, in_port=3, nw_src=10.0.0.5, actions=drop; rimuovendo la policy, h5 è tornato a 0% packet loss.

5. Valutazione finale e riproducibilità

Copertura dei requisiti

Requisito	Verifica	Risultato osservato
Modular design	OK	Responsabilità separate in moduli distinti.
Over-blocking	OK	h1 100% loss; h5 e h2 0% durante la mitigazione automatica.
Static threshold	OK	Training + EWMA; baseline 2.044 Mbit/s e threshold 3.066 Mbit/s.
Controller-centric decisions	OK	Blocklist JSON: regola priority=110 aggiunta e rimossa a runtime.
Inflexible block/unblock	OK	DROP attivo per 58.919 s e rimosso in base ai campioni di quiete.
Extra feature	OK	Allowlist runtime verificata tramite ALLOWLIST_SUPPRESSED.

Riproducibilità

```
cd ~/ncis-project && sudo mn -c
source ~/ryuenv/bin/activate
RUN_ID=FINAL_A ryu-manager --ofp-tcp-listen-port 6653 controller/dos_controller.py
sudo python3 topology/overblocking_topo.py
python3 scripts/generate_final_plots.py
```

I file usati per la validazione finale sono raccolti in results/final/: summary.csv, improvements.csv, raw/FINAL_A_controller.csv, raw/FINAL_B_controller.csv e i relativi grafici. Il branch finale del repository è branch-finale.